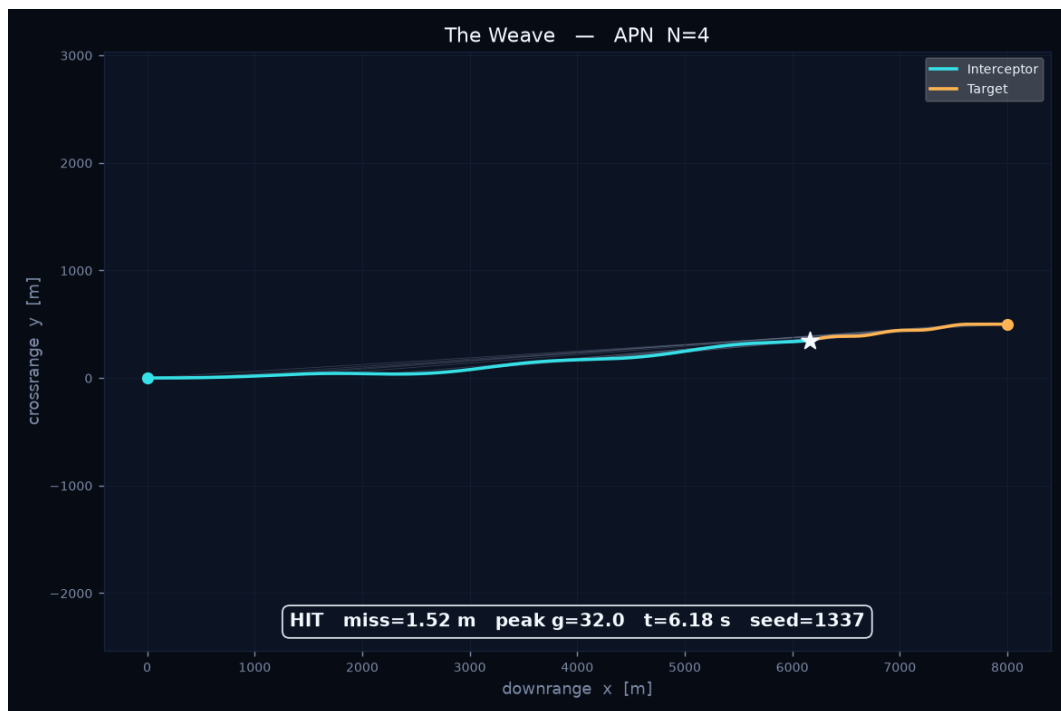


ZeroMiss

A validated missile-guidance interception simulator
Engineering Report



model -> validation -> Monte-Carlo results -> next steps

1. Problem & scope

Build a credible, demonstrable interception simulator for the terminal homing phase of a planar (3-DOF) engagement, covering the guidance-law family used across the field (pure pursuit, PPN, TPN, APN, ZEM-optimal), with realistic sensor and actuator imperfections, validated against published results, and packaged so a non-specialist can see it and a specialist can trust it.

Out of scope by design (see ETHICS.md): propulsion, mass/thrust, warhead/fuze, real seeker hardware, and any real platform or threat parameters. The interceptor is an idealized constant-speed point mass that can only turn; lethality is a single abstract radius. Everything is public-domain textbook math.

2. Model

Kinematics

constant-speed planar point mass; lateral accel turns the velocity vector

Integrator

fixed-step classical RK4 at 1 kHz ($dt = 1e-3$ s)

Guidance

$a = N * V_c * \lambda_{dot}$ (TPN) and the full PN family

Seeker

first-order LOS-rate lag + angular noise (+glint) + FOV/loss-of-lock + 100 Hz ZOH

Airframe

g-limit clamp + first/second-order autopilot lag

Targets

constant-velocity, step, weave, bang-bang, jink, scripted, live

Termination

parabolic-interpolated closest approach; lethal radius R_k

The load-bearing decision is fixed-step RK4 with a zero-order hold on the commanded and target accelerations: it makes runs bit-reproducible from a seed and lets an independent TypeScript re-implementation match the Python core to tolerance.

3. Verification & validation

All 7/7 checks run as green assertions (zeromiss validate) and as notebook figures:

[PASS] Zero miss (CV target, $N \geq 3$)

worst miss 0.364 m over Nxoffset grid (tol 0.5 m)

[PASS] Heading-error washout

worst miss 0.481 m (tol 0.5); latax decays=True

[PASS] Zarchan step-maneuver curve

peaks $N3/N4/N5 = 0.274/0.128/0.078$; peak@ $t/\tau=2$; ordering=True rises=True monotone=True settles=True

[PASS] Zarchan: nonlinear vs linear closed-form

linear peak@ $t/\tau=2.5$ (band True), sim peak@ $t/\tau=2$, $|?| \leq 1.5$ =True, ordering=True

[PASS] APN cancels a step TPN cannot

miss TPN=8.07 m, APN=0.196 m, ratio=0.024 (tol 0.1)

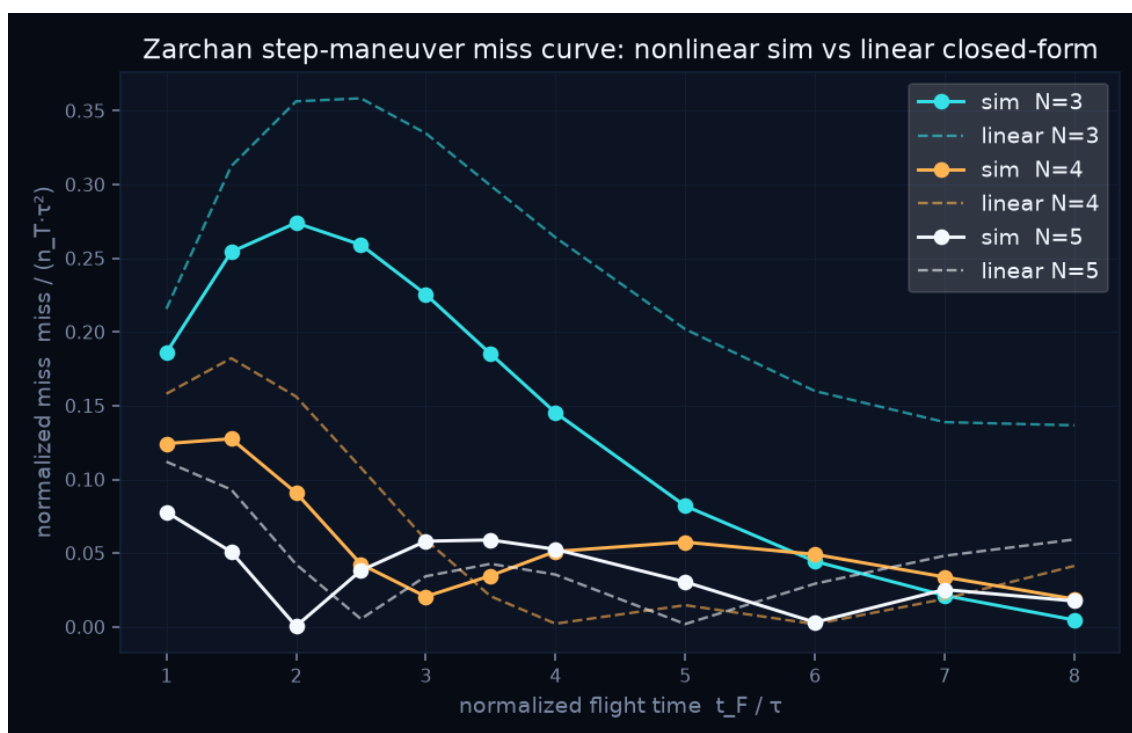
[PASS] RK4 order of accuracy

error ratio on /4 step = 255 (expect ~256, need >100)

[PASS] Step-convergence audit

$|\text{miss}(\text{dt}) - \text{miss}(\text{dt}/2)| = 0.0269$ m (tol 0.1 m)

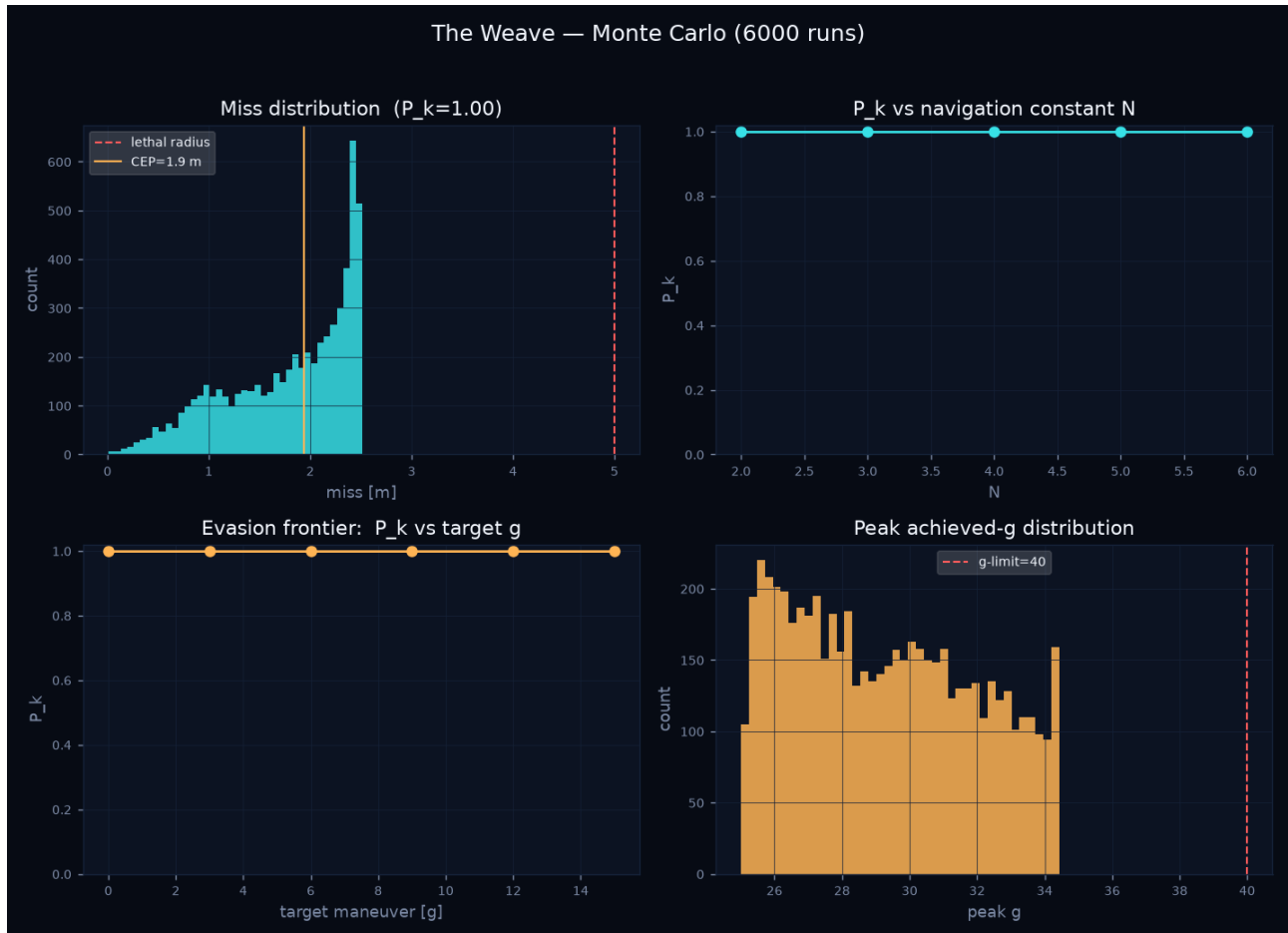
The headline: reproduced Zarchan step-maneuver curve



Plus the cross-engine check: the TypeScript twin reproduces every Python reference run to < 0.1% in CI. Independent re-implementation agreeing to tolerance is the strongest verification statement the project makes.

4. Monte-Carlo results

Campaigns randomize initial range, heading error, maneuver timing/magnitude, and seeker noise. The vectorized engine runs 10k-100k engagements in seconds and agrees with the scalar engine to machine precision on ideal runs. The headline metrics are P_k (probability of kill), CEP (median miss), and the miss / peak-g distributions; the trade-off studies are P_k -vs- N and the evasion frontier (P_k vs target g).



5. Software quality

Coverage

~95% line coverage on the engine; unit, property-based, validation, numerical tests

E2E

Playwright tests (desktop + mobile): launch->verdict, Duel, Compare, Storm, replay, reduced-motion, axe a11y

CI

lints + tests both engines, exports fixtures, runs cross-validation, builds the static app, runs E2E

Media

re-rendered from the engine in CI so it cannot drift from the code

6. What I'd do next

- 6-DOF attitude dynamics: the interceptor is a point mass today, with airframe rotation and control authority folded into one autopilot lag.
- Richer seeker error models: lag, angular noise, glint and a gimbal limit are modelled; clutter and multipath are not.
- Simulink in CI: the block-diagram leg is verified locally on R2024a and skips elsewhere, because automating it needs a licensed MATLAB runner.

7. Conclusion

ZeroMiss couples textbook-exact guidance laws to explicit sensor and actuator models, validates them against published results across five independent implementations, and reports performance statistically rather than anecdotally.